

C++

- * C++ is; cross-platform, General purpose, compiling, Object-Oriented, High level programming language.
- * Bjarne Stroustrup is the creator of C++.
- * C++ file extension is ".cpp".
- * C++ program has basic structure like this.

```
#include <iostream>
using namespace std;
int main() {
    return 0;
}
```

header file library for work with input and output objects.

This means that we can use names for objects and variables from the standard library.

Note: In the standard namespace library, the std keyword is used in the program.

Outputs

- * values output (print text) using cout object and insertion operator (<<) is used.
- * text is written in double quotes (" ") and escape character is used.

Ex: cout << "Hello World!";

cout → see-out
cin → see-in

- * cout objects are used to print text, etc.

Ex: cout << "Hello";
cout << "I love C++";

Output → Hello I love C++

① These outputs are given like this.

- * In the output, the escape character is used to print the text. It is called endl manipulator.

Ex: cout << "Hello World! \n";
cout << "Hello World! " << "\n";
cout << "Hello World! " << endl;
cout << "Hello World! ";

Output

Hello World!
Hello World!
Hello World!
Hello World!

* C++ හි ප්‍රධාන escape sequences කීහිහිනි.

Escape Sequence	Description
\n	new line
\t	creates horizontal tab
\\	backlash character
\"	Inserts a double quote character

* numbers print කිරීමේ දී double quotes භාවිතයට යොමු කළ හැකිය.
 උදා: cout << 3;
 cout << 3+3; Output 369
 cout << 3*3;

Comments

- * Single line comments කළයුතුය “//” යෙදවිය.
- * Multiline comments කළයුතුය “/* */” යෙදවිය.

Data types

* ප්‍රධාන data types 6 කි.

int $\Rightarrow$ 2 or 4 bytes $\rightarrow$ පූර්ව සංඛ්‍යා කළයුතුය

float $\Rightarrow$ 4 bytes $\rightarrow$ දශමයෙන් 6 ක් දක්වා දිගේ ප්‍රමාණ සංඛ්‍යා කළයුතුය

double $\Rightarrow$ 8 bytes $\rightarrow$ දශමයෙන් 15 ක් දක්වා දිගේ ප්‍රමාණ සංඛ්‍යා කළයුතුය

char $\Rightarrow$ 1 byte $\rightarrow$ තනි character / letter / number හෝ ASCII value එකක් කළයුතුය

bool $\Rightarrow$ 1 byte $\rightarrow$ true හෝ false value එකක් කළයුතුය

string $\Rightarrow$ 32 bytes $\rightarrow$ texts කළයුතුය

* bool type එකේ දී store කරනු ලබන්නේ true හෝ false යන values වලිනි,
 true $\rightarrow$ 1
 false $\rightarrow$ 0

ඉහත output කරයි.

* string data type එකේ ඇති කිරීමේ කළයුතුය <string> header file යොදාගනු ලැබේ.

```
#include <iostream>
#include <string>
using namespace std;
```

```
int main () {  
    string greeting = "Hello";  
    cout << greeting;  
    return 0;  
}
```

Output
=> Hello

Variables

* Variable creating syntax 2ଟା ପଦ୍ୟ 86ଟି.

```
datatype variableName = value;
```

```
int myNum = 15;  
float myFloat = 15.753;  
double myDouble = 2.65943;  
char myChar = 'A';  
string MyString = "Hello";  
bool isMarried = true;
```

* variable ଯାହା `int` କରେ `int`, ଏହାର value ଯାହା assign କରିବା ଆବଶ୍ୟକ ଅଟେ।
 ଉଦା: `int myNum;`

```

2204 int myNum;
      myNum = 15;
      cout << myNum;

```

Output
→ 15

Output $\Rightarrow 15$

* value re-assign କରିবো x এর মান ৫, যুক্ত।

```

20: int myNum = 10;
    myNum = 15;
    cout << myNum;

```

Output
=> 15

Output
=> 15

* text and variables combine with $\&$ operator called ampersand.

```
int myNum = 15;
```

```
int myNum = 15;
cout << "Here " << myNum << "is my variable.";
```

↓ output

Here 15 is my variable.

* නමුත් data type වෙනස් variables කිහිපයක් එකම මට්ටමක ඇත.

```
280:-) int x=5, y=6, z=7;
```

```
cout << x + y + z;
```

2) `int x, y, z;`

$$x = y = z = 50;$$

```
cout << x+y+z;
```

Identifiers

- * variables, functions, constants and identifiers are identifiers.
- * Identifiers naming rules are as follows:

- ★ Identifiers naming rules নির্দিষ্ট হয়।

(★) ଅଟେ letters, digits ଓ underscore (_) ମଧ୍ୟରେ ଥିବା ଅକ୍ଷର ।
 (★) ଅଟେ ଅକ୍ଷର ଓ ଅକ୍ଷର ।

(A) case-sensitive letters ଏବଂ underscore ଘଟେ ନାହିଁ।

④ case-sensitive

④ whitespaces, special characters નીચે સમાવેલા.

④ keywords ছেঁচি সোজা।

Constants

- * **unchangeable and read-only variables** නිර්වරණය කිරීමේ මෙය සිදු වේ.
- * **variable declaration** වෙනම 856 const keyword

* variable declaration `const` keyword එය යොදා.

```
const int myNum = 50;
```

```
const float PI = 3.14;
```

* variable କ୍ଷେତ୍ର declare କରିବା ପରେ ଏହାକୁ value କ୍ଷେତ୍ର assign କରା ଉଚିତ।

User Input

* User input **cin** object থেকে নেওয়া হয়।

* Shift extraction operator (>>) කෙරෙහි සැලකිය යුතුයි.

```

2204 int x;

```

cout << "Type a number : ";

```
cin >> x; // get user input
```

```
cout << "Your number is: " << x; // display input value
```

Operations

* ସ୍ଥଳ operators ରେଖା 6 କି.

⑤ Arithmetic operators

⑤ Assignment operators

④ Comparison operators

⑤ Logical operators

★ Bitwise operators

Arithmetic Operators

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value by 1	$++x$
--	Decrement	Decreases the value by 1	$--x$

Assignment Operators

Operator	Example	Same as
=	$x = 5$	$x = 5$
+=	$x += 5$	$x = x + 5$
-=	$x -= 5$	$x = x - 5$
*=	$x *= 5$	$x = x * 5$
/=	$x /= 5$	$x = x / 5$
%=	$x \% = 5$	$x = x \% 5$
&=	$x \& = 5$	$x = x \& 5$
=	$x = 5$	$x = x 5$
^=	$x ^ = 5$	$x = x ^ 5$
>>=	$x >> = 5$	$x = x >> 5$
<<=	$x << = 5$	$x = x << 5$

Comparison Operators

Operator	Name	Example
==	Equal to	$x == y$
!=	Not equal	$x != y$
>	Greater than	$x > y$
<	Less than	$x < y$
>=	Greater than or equal to	$x >= y$
<=	Less than or equal to	$x <= y$

Logical Operators

Operator	Name	Description	Example
&&	Logical AND	Returns true if both statements are true	$x < 5 \ \&\& \ x < 10$
	Logical OR	Returns true if at least one of the statements is true.	$x < 5 \ \ y < 5$
!	Logical NOT	Reverse the result true to false false to true	$!(x < 5)$

Strings

- * text store କିଛି strings କୋଡ୍‌ରେ. Double quotes ଆଉ text ଥିବା ପ୍ରକାଶିତ କରାଯାଏ.
string type ଥିବା ସହିତ କିଛି `<string>` header file ଥିବା କୋଡ୍‌ର ଉପରେ.
- * strings କିଛିକିଛି ଥିବା କାର, କିଛି string ଥିବା ଚେଷ୍ଟା ସହିତ କିଛି string
~~concat~~ concatenation ଚେଷ୍ଟା କରାଯାଏ.
- * string type variables ରେ ସହଜରେ concatenate କରା ଯାଏ.

```
for string firstName = "Araveen";  
string lastName = "Theekshana";  
string fullName = firstName + " " + lastName;  
cout << fullName;
```

⇒ output
Araveen Theekshana

string values $\Rightarrow$ ' + operator ' $\Rightarrow$ concatenate $\Rightarrow$ join together.

Ex:- cout << "Raveen" + " " + "Theekshana"; // Causes an Error!
insertion operator (<<) causes an error.

2201 cout << "Praveen" << " " << "Theekshana";
 ↓ output

Araveen Theekshana

* string append() function එක එකිනේ 2 වෙඩි බැස යාය.

```

220:- string firstName = "Praveen";
        string lastName = "Theekshana";
        string fullName = firstName.append(lastName);
        cout << fullName;

```

⇓ output

Praveen Theekshana

* numbers $\rightarrow$ strings
 Note: ' + operator ' works like "concatenation"

```

220+ 1) string x = "10";
      string y = "20";
      string z = x + y; // Outputs 1020 (a string)

```

```
2.) string x = "10";  
int int y = 20;  
string z = x + y; // Error
```

* string length / size (not memory size) এস ব্যবহার length() এবং size() functions ব্যবহার হয়.

```
220: string txt = "Hello";  
221: cout << txt.length() << endl;  
221: cout << txt.size();
```

Output
⇒

5
5

* string එකක ඇති characters access කිරීම indexing ක්‍රමය භාවිතය.

```
උදා: string myString = "Hello";  
cout << myString[0];
```

// Outputs H (first character)

(*) Indexing ක්‍රමයේ වැදගත් කරුණක් වන්නේ

⑧ Indexing ସୂଚକର ସହାୟତା 0 ରୁ 1 ର ମଧ୍ୟରେ।

* string ଥିବାରୁ ସ୍ତରୀକୃତ character ରେ output କରିବି ଏହା ସ୍ପଷ୍ଟ ହେଉଛି ନାହିଁ.

```
ex:- string myString = "Hello";  
cout << myString[myString.length() - 1];  
// Outputs o
```

* string modify કરી શકાય છે, indexing સરળ છે.

Ex: string myString = "Hello";

myString[0] = 'J'; Output Jello
cout << myString; ↑

① જો string modify કરવા character થી શરૂ કરવામાં આવે તો single quotes જરૂરી છે. Double quotes જરૂરી નથી, error થાય છે.

* string modify કરવા at() function વાપરવામાં આવે છે, string modify કરે શકાય.

Ex: string myString = "Hello";

cout << myString.at(0) << endl; // Outputs first character

myString.at(0) = 'J';

cout << myString; // Outputs Jello

* User input થી string લેવા માટે string લેવાની જરૂર છે.

Ex: string firstName;

cout << "Type your first name: ";

cin >> firstName;

cout << "Your name is: ", firstName;

① જો space / tab space જેવા terminating character લેવામાં આવે તો અસર થાય છે.

space / tab space જેવા terminating character લેવામાં આવે તો અસર થાય છે.

* string લેવા માટે getline() function વાપરવામાં આવે છે. જે cin → first parameter થી લે છે, string variable → second parameter લે છે, જેને ફોર્મટ કરે છે.

Ex: string fullName;

cout << "Type your full name: ";

getline(cin, fullName);

cout << "Your name is: " << fullName;

* standard namespace library થી C++ programs run કરવા માટે std keyword સાથે :: operator વાપરવામાં આવે છે.

// using namespace std; - Remove this line

int main () {

std::string greeting = "Hello";

std::cout << greeting;

return 0;

}

* string type માંથી બનાવે, char type array યોગ્ય રીતે strings store કરે શકે.

```
જેમ: char greeting[] = "Hello";  
cout << greeting;
```

Output $\Rightarrow$ Hello

⊛ C-style strings રીતે જાણે.

Math functions

① max(x,y) and min(x,y)

* શ્રેષ્ઠ value એક અને શ્રેષ્ઠ value એક જાણી શકે.

```
જેમ: 1) cout << max(5,10); } 2) cout << min(5,10);
```

⊛ * ગણતરી math functions માટે <cmath> header file યોગ્ય કરે.

```
જેમ: #include <cmath>  
cout << sqrt(64);  
cout << round(2.6);  
cout << log(2);
```

⊛ sqrt(x) $\rightarrow$ square root

round(x) $\rightarrow$ rounds a number

log(x) $\rightarrow$ natural logarithm

if - else if - else

* બે syntax એક જાણી શકે.

```
if (condition1) {  
    // block of code to be executed  
}  
else if (condition2) {  
    // block of code to be executed  
}  
else {  
    // block of code to be executed  
}
```

Shorthand if - else (Ternary Operator)

* બે syntax એક જાણી શકે.

```
variable = (condition) ? expressionTrue : expressionFalse;
```

Switch

★ જોઈ syntax રીતે જણાવે છે.

```
Switch (expression) {  
    case x :  
        //Code block  
        break;  
    case y :  
        //Code block  
        break;  
    default :  
        //Code block  
}
```

Loops

while loop

★ જોઈ syntax રીતે જણાવે છે.

```
while (condition) {  
    //Code to be executed  
}
```

★ loop body રીતે જુદું increment અને decrement operation રીતે થઈ શકે છે.
જાણજોનાર; infinite loop રીતે જાણે.

do-while loop

★ જોઈ syntax રીતે જણાવે છે.

```
do {  
    //Code to be executed  
}  
while (condition);
```

★ જોઈ રીતે increment અને decrement operation રીતે થઈ શકે છે.

For loop

★ જોઈ syntax રીતે જણાવે છે.

```
for (statement 1; statement 2; statement 3) {  
    //Code to be executed  
}
```

```

for (int i=0; i<5; i++) {
    cout << i << "\n";
}

```

Output
⇒

0
1
2
3
4

foreach loop

- ★ લૉપ એક array એક elements કરતા loop થી કરવામાં.
- ★ લૉપ syntax એક જમણા ઢાંચે છે.

```

for (type variableName : arrayName) {
    // Code to be executed
}

```

Introduced in
C++ version 11
(2011)

Not working
on Quincy

```

for (int i : myNumbers) {
    cout << i << "\n";
}

```

⇒
Output

10
20
30
40
50

break / Continue

- ★ જ્યારે કોઈ code block એક execution એક પૂરું થાય છે, તો code block એક break keyword વાપરે છે.
- ★ code block એક iteration એક (especially in loops) skip કરે છે.

```

1) for (int i=0; i<10; i++) {
    if (i==4) {
        break;
    }
    cout << i << "\n";
}

```

```

2) for (int i=0; i<10; i++) {
    if (i==4) {
        continue;
    }
    cout << i << "\n";
}

```

Arrays

- ★ લૉપ syntax એક જમણા ઢાંચે છે.

```

data type    arrayName[] = { member1, member2, ... };

```

- ★ array length એક કહેવામાં આવે છે; array length એક કહેવામાં આવે છે.
- ★ array elements access કરે છે અને modify કરે છે indexing વાપરે છે.

* Array એક્સ 'declare' કરતા પહેલાં elements હોવા જોઈએ, અથવા હોવાની અનિશ્ચિતતા હોવી.

```
Ex:- string cars[5];  
cars[0] = "Volvo";  
cars[1] = "BMW";  
cars[2] = "Ford";  
cars[3] = "Mazda";  
cars[4] = "Tesla";
```

Ⓐ જે પ્રકારના elements હોવા જોઈએ તે; declare કરતાં જ array length એક હોય જોઈએ!

* C++ માં array એક જ size એક જ fixed છે. તે એક્સ; array elements modify કરી શકીએ, add કરીએ એમ remove કરીએ આ શક્ય નથી.

* array elements add કરીએ એમ remove કરીએ શક્ય એક જ પ્રકારના એક્સ vectors એક્સ હોય છે. જે resizable arrays હોય છે, એક્સ.

* જે એક્સ એક્સ એક્સ <vector> library header file એક્સ એક્સ.

* Array size એક્સ એક્સ sizeof() operator એક્સ.

```
Ex:- int myNumbers[5] = {10, 20, 30, 40, 50};  
cout << sizeof(myNumbers);
```

→ 20
Output

Ⓐ જે એક્સ array length એક્સ 5 એક્સ, 20 output એક્સ;
int type variable એક્સ → 4 bytes
array એક્સ int values → 5
 $4 \times 5 = 20 \text{ bytes}$ એક્સ.

જે એક્સ array length એક્સ એક્સ એક્સ એક્સ એક્સ.

```
Ex:- int myNumbers[5] = {10, 20, 30, 40, 50};  
int getArrayLength = sizeof(myNumbers)/sizeof(myNumbers[0]);  
cout << getArrayLength;
```

↓ Output

5

* Multidimensional arrays declare કરીએ એક્સ data type એક્સ array name એક્સ, એક્સ એક્સ square brackets એક્સ main array એક્સ elements એક્સ, એક્સ square brackets એક્સ sub array elements એક્સ એક્સ.

```

2201-1) string letters [2][4] = {
    {"A", "B", "C", "D"},
    {"E", "F", "G", "H"}
};

```

```

2) string letters [2][2][2] = {
    {
        {"A", "B"},
        {"C", "D"}
    },
    {
        {"E", "F"},
        {"G", "H"}
    }
};

```

* we can access array elements and modify them using indexing syntax.

```

2201-1) cout << letters[0][2];

```

```

2) letters[0][2] = "Z";
   cout << letters[0][2];

```

Structures

* we can declare several variables of different data types using structures.

* Structure is created using struct keyword.

Structure members, curly braces are used.

declare a structure variable using variable name.

```

2201- struct {
    int myNum;           // int member
    string myString;     // String member
} myStructure;          // Structure variable

```

* Structure members are accessed using dot syntax.

```

2201- myStructure.myNum = 1;

```

```

   cout << myStructure.myNum;

```

- * variable නිවැරදිව ලියන විට structure වලට ස්ථාන ලබාදීමට, comma separated list වලින් ලියන variable names සලකයි.

```

උදා: struct {
    int myNum;
    string myString;
} myStruct1, myStruct2; // Multiple structure variables

myStruct1.myNum = 1;
myStruct2.myNum = 2;

```

- * ඉහත structure වලට නමක් ලබාදීමේ හැකියාව ඇත. එනම්, එය named structure ලෙස හැඳින්වේ. මෙය මෙම මත data type වලින් ලියන හැකියාව ලබාදේ.

මෙම structure වලට ස්වයංක්‍රීයව program වලින් විවිධ නම් ලබාදීමේ හැකියාව ඇත.

```

උදා: struct myDataType { // This structure named "myDataType"
    int myNum;
    string myString;
};

```



```

myDataType myVar; // Declare a variables using previous
                    structure name as the data type.

```

Enumeration

- * Constants ලෙසින් විස්තරයක් සහිතව enum ස්වයංක්‍රීයව ලබාදේ.
- * එහි මගින් enum වලින් විවිධ නම් ලබාදීමේ හැකියාව ඇත.

```

enum Level {
    LOW,
    MEDIUM,
    HIGH
};

```

- * enum වලින් access කිරීමට, ඊට අදාළ variable වලින් විවිධ නම් ලබාදීමේ හැකියාව ඇත.

```

උදා: enum Level myVar;

```


enum variable ଥିବାର value ଥିବାର assign କାରି ହାତୀ. ତେଣୁ value ଥିବା enum ଥିବାର item ଥିବାର ବିଧି ଉପସ୍ଥାପନ.

ଉଦା: enum Level myVar = MEDIUM;

* default ରେକ୍ଟ enum ଥିବାର କେଉଁ item ଥିବାର value ଥିବା 0, କେଉଁ item ଥିବାର value ଥିବା 1 ମୁତାବକ ରେକ୍ଟ କରାଯିବ.

① ଯଦି ଉପରୋକ୍ତ myVar print କାରି ହେବାର, output ରାଉଟ 1 ଥିବ.

ଉଦା: cout << myVar; Outputs 1

* enum items ଥିବାର କେଉଁ କେଉଁ values ଥିବାର ଦିଆଯିବାର ୧, ୨, ୩.

ଉଦା: enum Level {

LOW = 5;

MEDIUM = 10;

HIGH = 20;

};

* specific item ଥିବାର value ଥିବାର ମୁତାବକ, ତେଣୁ କେଉଁ କେଉଁ values ରେକ୍ଟ କାରି ହେବ.

ଉଦା: enum Level {

LOW; // Now this is 0

MEDIUM = 6; // This is 6

HIGH; // This is 7

};

References

* କେଉଁ reference variables ବ୍ୟବହାର କାରି ହାତୀ. ଥିବାର, ଯଦି କେଉଁ variable ଥିବାର reference ଥିବା ରେକ୍ଟ କରାଯିବ.

* reference variables ବ୍ୟବହାର କରାଯିବାର '& operator' ବ୍ୟବହାର.

ଉଦା: string food = "Pizza";

string &meal = food; //reference to food variable

cout << food; //Outputs Pizza

cout << meal; //Outputs Pizza

Memory Address

- * '& operator' એ reference variables ચિહ્નિત કરે છે, memory addresses નો ભાગ છે, બેસ્ટ બનેલો.

```
જાન્યુ ૨૦૨૦ string food = "Pizza";
```

```
cout << &food; // Outputs 0x.....
```

- ④ Memory Address એ લેખકોને hexadecimal (અર્થાત્) value એકાદી બતાવે.

Pointers

- * Pointer એક variable નું memory address એક value એક assign કરે છે, variable એક.
- * pointer data type એક memory address એક variable data type એક જોડે છે. '* Operator' બેસ્ટ.

```
જાન્યુ ૨૦૨૦ string food = "Pizza";
```

```
string* ptr = &food;
```

```
cout << ptr; // Outputs 0x.....
```

- ④ Pointer variables declare કરે છે અને ચિહ્નિત કરે છે.

```
string* myString; // ચિહ્નિત કરે છે
```

```
string *myString;
```

```
string * myString;
```

Dereferencing

- * '* operator' એ "dereference operator" બેસ્ટ છે, ચિહ્નિત કરે છે.
- * pointer એક memory address એક ભાગ, value એક ભાગ ભાગે છે.

```
જાન્યુ ૨૦૨૦ string food = "Pizza";
```

```
string* ptr = &food;
```

```
cout << ptr; // Outputs memory address
```

```
cout << *ptr; // Outputs Pizza
```

- * Pointer value એક modify કરે છે, એક ભાગ બેસ્ટ કરે છે. એક ભાગ રહે variable એક value એક modify થી જીવે.

```

2201- string food = "Pizza";
      string* ptr = &food;

      *ptr = "Hamburger";

      cout << *ptr << endl;
      cout << food << endl;

```

⇒
Output
Hamburger
Hamburger

Functions

- * Function එකක් නිර්මාණය කිරීමේ return type, function name and parentheses සහිත යුතුය.

```

2202- void myFunction () {
      // Code to be executed
}

```

⊙ void වර්ග
ප්‍රකාශනය; මෙම
function එක අපේ
return value එකක්
සපයන්නේ නැත.

- * Function එකක් run වීමට හේතු වන main() method එක ඇති call කළ යුතුය. මෙය ඉවහර වර්ගයෙන් call කළ හැක.

```

2203- void myFunction () {
      cout << "I just got executed";
}

int main () {
      myFunction(); // Call the function
      return 0;
}

```

- * C++ function එකක් කොටස් 2 කි.

- (i) Declaration ⇒ the return type, name of the function and parameters (if any)
- (ii) Definition ⇒ body of the function (code to be executed)

- * main() ට එකම function එකක් declare කළහොත් error එකක් ලැබේ.
- * මෙම ගැටලුවට විසඳුමක් ලෙස;

- ⊙ function declaration එක main() ට ඉහළින් කොට,
- ⊙ function definition එක main() ට පහළින් සහිත කළ.

මෙම ක්‍රමය ඉගෙනගන්නේ more readable and well organized code එකක් ලබාගත හැක.

උදා:- void myfunction (); // function declaration

```
int main () {  
    myfunction ();
```

```
    return 0;  
}
```

```
void myfunction () { // function definition  
    cout << "I just executed";  
}
```

* Functions තුළ parameters ලෙස නොරැහැරූ දෑ තුළින් කළ හැක. parameters ක්‍රියා කරන්නේ; function එක තුළ ව variables ලෙසයි.

* ඉවසිය යුතු parameters , comma separated list එකින් ලෙස parentheses තුළට සෙවිය හැක.

උදා:- void myfunction (string fname, string lname) {
 cout << fname << " " << lname << endl;
}

```
int main () {  
    myfunction ("Praveen", "Theekshana");  
    return 0;  
}
```

⇓ output

Praveen Theekshana

⊗ මෙම දෑ රැහැරුවේ fname හා lname ⇒ parameters වේ.

"Praveen" හා "Theekshana" ⇒ Arguments වේ.

* Parameter එකකට default value එකක් ලියැවිය හැක. මේවා; ඉන් එකින් argument එකක් ලෙස නොලැබූ ඉවසිය යුතු argument එක ලෙස ක්‍රියා කරන්නේ මෙම default value එකයි.

උදා:- void myfunction (string country = "Sri Lanka") {
 cout << country << endl;
}

```
int main () {  
    myfunction ("India");  
    myfunction ();  
    return 0;  
}
```

⇒
Output
India
Sri Lanka

function એકાંતિ return value એકાંતિ આ બાબતો ધ્યાન રાખી; void એકાંતિ
 ધ્યાન return type એકાંતિ (int, string) એકાંતિ હોય. એકાંતિ; function એકાંતિ
 return keyword એકાંતિ હોય.

```

    ૧૧૦- int myfunction(int x) {
        return 5+x;
    }
    
```

```

    int main () {
        cout << myfunction(3);
        return 0;
    }
    
```

Output
 ⇒ 8

* આ એકાંતિ parameters એકાંતિ ધ્યાન એકાંતિ આકાંતિ એકાંતિ, result એકાંતિ variable
 એકાંતિ store એકાંતિ આકાંતિ ૨ ધ્યાન.

```

    ૧૧૦- int myfunction (int x, int y) {
        return x+y;
    }
    
```

```

    int main () {
        cout << myfunction (5, 3) << endl;
        return 0;
        int z = myfunction (5, 3);
        cout << z;
        return 0;
    }
    
```

8
 ⇒ 8
 Output

* Parameter એકાંતિ એકાંતિ normal variables એકાંતિ references pass એકાંતિ
 એકાંતિ ધ્યાન. argument values change એકાંતિ ધ્યાન ધ્યાન એકાંતિ એકાંતિ
 એકાંતિ એકાંતિ એકાંતિ.

```

    ૧૧૦- void swapNums (int &x, int &y) {
        int z = x;
        x = y;
        y = z;
    }
    
```

```

    int main () {
        int firstNum = 10;
        int secondNum = 20;

        cout << "Before swap : " << firstNum << secondNum << endl;
        swapNums (firstNum, secondNum);

        cout << "After swap : " << firstNum << secondNum << endl;
        return 0;
    }
    
```

* C++ లలో ఒకే పేరు ఉన్న, వేర్వేరు parameters గల functions నిర్వచించుట అనేది function overloading అని పిలుస్తారు.

```

// C++ code for function overloading
int plusfunc(int x, int y) {
    return x+y;
}

double plusfunc(double x, double y) {
    return x+y;
}

int main () {
    int myNum1 = plusfunc(8,5);
    double myNum2 = plusfunc(4.3, 6.25);
    cout << "Int: " << myNum1 << endl;
    cout << "Double: " << myNum2 << endl;
    return 0;
}

```

* ఒకే variable name లోని variables లెక్క; ఇలాంటి functions లోని (global variable) ఇది లోకల్ function లోని (local variable) తేడాను తెలుపుతుంది. ఇది C++ లోని ఒక ప్రత్యేక లక్షణం.

```

// C++ code for variable scope
int x = 5; // Global variable x

void myfunction () {
    int x = 22; // Local variable x
    cout << x << endl;
}

int main () {
    myfunction (); // refers to local variable x -> 22
    cout << x; // refers to global variable x -> 5
    return 0;
}

```

(A) Note: ఒకే పేరు ఉన్న, వేర్వేరు variables లెక్క; ఇలాంటి functions లోని (global variable) ఇది లోకల్ function లోని (local variable) తేడాను తెలుపుతుంది. ఇది C++ లోని ఒక ప్రత్యేక లక్షణం.

Recursion

* କିଛି function ଥିବାକୁ ଆପଣ ବିକଳରେ ଆପଣ call କରିବା ସମୟରେ ଫିର ଲେଖି ଆସୁଥିବେ. ତଥା ବିକଳର ଗୋଟିଏ କ୍ଷୁଦ୍ର କୋଡ୍ ଗୋଟିଏ ଥର ଥର, ପରସ୍ପରକୁ ବିକଳି ଆସିବା ଗୋଟିଏ କ୍ଷୁଦ୍ର କୋଡ୍ ଥିବା ଥିବା.

```
int sum(int k) {  
    if (k > 0) {  
        return k + sum(k-1);  
    }  
    else {  
        return 0;  
    }  
}  
  
int main() {  
    int result = sum(10);  
    cout << result;  
  
    return 0;  
}
```

How it works

* Think firstly $k=10$; Then,

$$10 + \text{sum}(9)$$

$$10 + (9 + \text{sum}(8))$$

$$10 + (9 + (8 + \text{sum}(7)))$$

⋮

$$10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + \text{sum}(0)$$

$$10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 0$$